MOUNTAIN CAR

Reinforcement Learning Project — Theoretical Report

Assignment 4: Controlling Mountain Car (`MountainCar-v0`)

Course: Artificial Intelligence

Environment: MountainCar-v0

Approach: Tabular Q-learning with a discretized state space

Team name: [Enter team name]

Team members and registration numbers:
1. [Full name — Registration number]
2. [Full name — Registration number]
3. [Add or remove rows as needed]

Complete the team details and attach the executed Python notebook. This PDF presents the theoretical component; the notebook is the programming component.

1. Introduction

Mountain Car is a classic control problem used to study how an agent learns sequential decisions by interacting with an environment. A small car is positioned in a valley between two hills. Its engine is not powerful enough to drive directly up the hill to the target. The car must first move away from the target, build momentum, and then use that momentum to climb the opposite hill.

The task is formulated as reinforcement learning. The agent observes the car's position and velocity, chooses an action, receives a reward, and observes the next state. Through repeated interaction, it learns a policy for selecting actions.

This project uses the standard Gymnasium environment `MountainCar-v0` and tabular Q-learning. The supplied starter file `environment.py` specifies 40 position bins, 40 velocity bins, three actions, a 200-step limit and a zero-initialized Q-table.

2. Problem statement and objectives

The problem is to learn a control policy that moves the car to the goal position within the episode limit. The agent must learn when to push left, remain neutral or push right, despite receiving a simple time-step penalty.

- Describe Mountain Car as a reinforcement learning problem.
- Identify observations, actions, reward and episode-ending conditions.
- Convert continuous position and velocity into discrete state indexes.
- Represent state-action values using a Q-table.
- Implement Q-learning with epsilon-greedy exploration.
- Evaluate the learned policy over multiple episodes and report measured performance.

3. MountainCar-v0 environment

3.1 State or observation

The environment returns two continuous values: **position** and **velocity**. Position describes where the car is along the track, while velocity describes its direction and speed. Both are needed because two observations with the same position but different velocities may require different actions.

For the standard MountainCar-v0 environment, position ranges approximately from -1.2 to 0.6 and velocity from -0.07 to 0.07. The goal is near position 0.5.

3.2 Action space

Action	Meaning
0	Push left
1	Do not push / neutral
2	Push right

3.3 Reward and episode ending

The standard task gives a reward of -1 at each time step. Reaching the goal sooner therefore accumulates fewer negative rewards. An episode ends when the car reaches the goal (`terminated=True`) or when the 200-step time limit is reached (`truncated=True`). These conditions are distinct and should be handled separately.

The implementation retains the environment's reward. It does not introduce reward shaping, so the agent learns on the assigned task's original reward structure.

4. Reinforcement learning formulation

Reinforcement learning is a learning approach in which an agent learns by interacting with an environment. At each time step, the agent observes a state, selects an action, receives a reward and moves to a new state. Its objective is to learn a policy that maximizes expected cumulative reward.

- **Agent:** the learning controller selecting the push direction.
- **Environment:** MountainCar-v0, which simulates the car and track.
- **State:** the discretized representation of position and velocity.
- **Action:** push left, neutral or push right.
- **Reward:** -1 per time step in the standard task.
- **Policy:** the rule used to select an action in a state.
- **Return:** accumulated discounted reward over an episode.

Q-learning is model-free: the agent does not need a programmed model of the car's physics. It learns value estimates from transitions sampled by interacting with the environment.

5. State-space discretization

The observation is continuous, but a tabular algorithm stores values in a finite table. The supplied `environment.py` divides position into 40 bins and velocity into 40 bins. It uses `numpy.linspace` to create 39 internal boundaries per dimension and `numpy.digitize` to map observations to integer indexes.

Setting	Value	Purpose
Position bins	40	Discretizes position
Velocity bins	40	Discretizes velocity
Actions	3	One value per available action
Maximum steps	200	Episode limit in starter file

The indexes range from 0 to 39, matching the table dimensions. Discretization is an approximation: continuous observations within the same bin are treated as one discrete state. The notebook clips indexes to the valid range as a defensive safeguard.

6. Q-table representation

A Q-table stores the estimated quality of taking action a in state s. There is one value for every combination of position bin, velocity bin and action.

```
40 position bins × 40 velocity bins × 3 actions = 4,800 Q-values
```

The starter file initializes the table with zeros. Initially the agent has no learned preference among actions. The learning process updates these values using rewards and estimates of future value.

7. Q-learning algorithm

Q-learning is a model-free temporal-difference algorithm. It updates the current state-action estimate using the reward observed and the highest estimated action value in the next state.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max Q(s',a') - Q(s,a)]$$

- **Q(s,a):** estimated value of taking action a in state s.
- α **(alpha):** learning rate, controlling the size of an update.
- **r:** immediate reward received.
- γ **(gamma):** discount factor for future rewards.

- **s':** next state after the action.
- **max Q(s',a'):** best estimated action value in the next state.

For a true terminal transition, there is no future value to add, so the target is the immediate reward. A time-limit truncation ends data collection for the episode but is not the same as reaching the task's goal. The notebook preserves this distinction.

8. Exploration and exploitation

Exploration means trying actions to collect information. Exploitation means choosing the action currently estimated to be best. The notebook uses epsilon-greedy action selection:

- With probability ε, choose a random action (exploration).
- With probability $1 - \varepsilon$, choose an action with the highest Q-value (exploitation).
- If several actions share the maximum value, choose randomly among them.

Epsilon starts high so that the agent explores, then decays toward a minimum so later training emphasizes learned values. The notebook starts with $\varepsilon = 1.0$, minimum $\varepsilon = 0.01$ and multiplicative decay 0.9995 per episode.

9. Training and evaluation

Training repeats episodes. The environment is reset, the initial observation is discretized, and the agent repeatedly selects actions, receives rewards, discretizes next observations and updates the Q-table. The episode stops when terminated or truncated. Epsilon is reduced after each episode.

The notebook starts with 15,000 episodes, learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.99$. These are experimental starting settings, not a guarantee of a particular success rate. Training progress is displayed and plotted using 100-episode moving averages.

After training, the policy is evaluated over 100 separate episodes with epsilon set to zero. The notebook reports mean reward, mean episode length and success rate. Numerical results must come from running the notebook; this report does not invent a performance result.

10. System workflow

1. Start MountainCar-v0.

2. Receive position and velocity.

3. Discretize the observation into two indexes.

4. Choose an action using epsilon-greedy selection.

5. Apply the action and receive reward and next observation.

6. Update the Q-table using the Q-learning rule.

7. Check whether the episode terminated or was truncated.

8. Repeat until the episode ends; decay epsilon and start the next episode.

9. Evaluate the greedy policy over multiple episodes.

10. Plot learning progress and save the Q-table.

Training updates the Q-table. Evaluation keeps it fixed, which makes the evaluation a measurement of the learned policy rather than additional training.

11. Project constraints and assumptions

- The implementation targets the standard `MountainCar-v0` environment.
- The starter file specifies a maximum episode length of 200 steps.

- The state representation uses position and velocity only.
- The table uses 40 bins per observation dimension and three actions.
- The Q-table is initialized to zero and stored as a NumPy array.
- The notebook uses Gymnasium's five-value `step()` API.
- The environment reward is retained; no reward shaping is added.
- Results depend on hyperparameters, software versions and random variation; performance must be measured.

12. Review of the supplied starter files

12.1 environment.py

The supplied file defines the environment name, maximum steps, discretization sizes and action count. It provides `make_bins()` to construct boundaries, `get_state()` to map an observation to two indexes, and `create_q_table()` to allocate a zero-filled table. These definitions are consistent with tabular Q-learning.

The notebook adds defensive clipping of indexes to the valid range and explicitly converts them to integers. This prevents an observation at a boundary from producing an invalid table index.

12.2 .gitignore

The supplied `.gitignore` excludes `.venv/`, `__pycache__/`, `*.pyc` and `q_table.npy`. These are local environment or generated artifacts. Since `q_table.npy` is ignored, it will normally not be committed to Git. If the lecturer requires the trained table, submit it separately or deliberately revise the ignore rule.

Only `environment.py` and `.gitignore` were available for this review. The complete original training implementation and any additional project files were not supplied. Therefore, this report reviews the supplied starter files and the notebook implementation; it does not claim to audit unseen files.

13. Limitations and possible improvements

- **Discretization loss:** binning discards precision and can group observations that need different actions.
- **Hyperparameter sensitivity:** learning rate, discount factor, epsilon schedule and training duration affect results.
- **Run-to-run variation:** repeat experiments with multiple seeds for stronger evidence.
- **Tabular scalability:** tables become impractical for large or high-dimensional state spaces.
- **Evaluation scope:** 100 episodes estimate performance but do not prove perfect behavior.
- **Rendering dependency:** a graphical display may not be available in hosted notebooks.

Future work could compare bin counts and hyperparameters, evaluate multiple random seeds, or investigate function approximation. These are possible extensions, not features claimed as already implemented.

14. Conclusion

Mountain Car demonstrates how reinforcement learning can solve a control task through repeated interaction rather than a hand-written action sequence. This project uses discretized position and velocity, a Q-table containing 4,800 state-action values, Q-learning updates and epsilon-greedy exploration. A separate greedy evaluation measures whether the learned policy reaches the goal and how many steps it takes.

The supplied `environment.py` provides the basic discretization and Q-table structure. The accompanying notebook completes the training, evaluation, visualization and saving workflow. Final performance claims should be based on the actual output produced when the notebook is run.

15. References

- Gymnasium documentation, Classic Control: Mountain Car: https://gymnasium.farama.org/environments/classic_control/mountain_car/

- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press.
- NumPy documentation: `linspace`, `digitize`, `zeros`, `save` and `load`.

End of theoretical report.